

Guía Explicativa: Ejercicios de Listas y Pilas (3, 6, 9, 12, 15 y 17)

Este documento ofrece una explicación conceptual y detallada del funcionamiento, algoritmos y diseño de los 6 ejercicios resueltos de la guía de **Listas y Pilas** en la materia de Algoritmos y Estructura de Datos.

Índice

- Guía Explicativa: Ejercicios de Listas y Pilas (3, 6, 9, 12, 15 y 17)
 - Índice
 - Ejercicio 3: Lista con Arreglos de Enteros y Eliminación de Ocurrencias
 - Enunciado
 - Concepto y Algoritmo
 - Ejercicio 6: Simulación de Lista Circular de Caracteres
 - Enunciado
 - Concepto y Algoritmo
 - Ejercicio 9: Suma de Elementos de una Lista
 - Enunciado
 - Concepto y Algoritmo
 - Ejercicio 12: Eliminar Valores Repetidos de una Lista
 - Enunciado
 - Concepto y Algoritmo
 - Ejercicio 15: Inversión de Lista Enlazada (Iterativo y Recursivo)
 - Enunciado
 - Concepto e Implementación
 - A) Algoritmo Iterativo (`reverseIterative`)
 - B) Algoritmo Recursivo (`reverseRecursive`)
 - Ejercicio 17: Inversión de una Pila mediante Recursión
 - Enunciado
 - Concepto y Algoritmo
 - Algoritmo de Inversión (`invertir`)
 - Algoritmo Auxiliar de Inserción en el Fondo (`insertAtBottom`)

Ejercicio 3: Lista con Arreglos de Enteros y Eliminación de Ocurrencias

Archivo de código: [03-EliminarN.cpp]

(file:///c:/Users/AARON/Documents/GitHub/Guia_Ejercicios/Algoritmos_Y_Estructura_De_Datos/5-Listas_Y_Pilas/03-EliminarN.cpp)

Enunciado

Dada una lista con arreglos de enteros y un número entero n , implementar una función donde se eliminen todos los números n de la lista.

Concepto y Algoritmo

En esta estructura de datos, cada nodo de la lista enlazada no contiene un simple entero, sino un **arreglo dinámico de enteros** (`int* datos`) y su tamaño asociado (`int tamaño`). El problema nos pide buscar y remover todas las ocurrencias del número n dentro de todos los arreglos contenidos en la lista.

Pasos del Algoritmo:

1. Recorremos cada nodo de la lista desde el primer nodo de datos (`first()`) hasta el centinela final (`tail`).
2. Para cada nodo, verificamos si su arreglo contiene el valor n .
3. Si lo contiene, contamos cuántas veces aparece para calcular el tamaño del nuevo arreglo:
$$\text{NewSize} = \text{OldSize} - \text{Ocurrencias de } n.$$
4. Creamos un nuevo arreglo dinámico en memoria con el nuevo tamaño.
5. Copiamos todos los elementos del arreglo antiguo que **no sean iguales a n** al nuevo arreglo.
6. Liberamos la memoria del arreglo antiguo (`delete[] datos`) y reasignamos el puntero del nodo al nuevo arreglo.
7. Ajustamos el atributo `tamaño` del nodo al nuevo tamaño.

Visualización Conceptual:

```
Nodo Original: [ Tamaño: 5 | Datos: [3, 5, 2, 5, 8] ] -> Siguiende
Remover (5) : Nuevo tamaño calculando = 5 - 2 = 3.
Nodo Filtrado: [ Tamaño: 3 | Datos: [3, 2, 8] ] -> Siguiende
```

Ejercicio 6: Simulación de Lista Circular de Caracteres

Archivo de código: [06-SimularCircular.cpp]

(file:///c:/Users/AARON/Documents/GitHub/Guia_Ejercicios/Algoritmos_Y_Estructura_De_Datos/5-Listas_Y_Pilas/06-SimularCircular.cpp)

Enunciado

Implemente una lista con arreglo que simule una lista circular de caracteres con n posiciones y, dados dos enteros m e i , imprima m valores a partir de la posición i .

Concepto y Algoritmo

Una lista circular simplemente enlazada es una lista donde el último nodo apunta de regreso al primer nodo de datos (o al nodo cabecera). En este ejercicio, en lugar de utilizar un puntero `tail` que apunte a `nullptr`, se establece la circularidad haciendo que `head->next = head` en una lista vacía, y al insertar nodos, el último nodo físico mantiene su enlace apuntando a `head`.

Pasos del Algoritmo para Imprimir:

1. Para imprimir a partir de la posición lógica i , calculamos el índice inicial real dentro de la lista utilizando aritmética modular: $\text{Posición Inicial} = i \pmod{\text{size}}$. Esto evita desbordamientos si i es mayor que

el tamaño de la lista.

2. Avanzamos en la lista desde el primer elemento tantas veces como indique la posición inicial calculada.
3. Imprimimos el nodo y avanzamos al siguiente. Repetimos este paso m veces.
4. Gracias a la circularidad física del enlace **next**, cuando el recorrido llega al final del ciclo de datos, salta automáticamente al nodo centinela **head** y de ahí de regreso al primer nodo, permitiendo un recorrido infinito y circular.

Visualización Conceptual:



```
[ head ] -> [ "A" ] -> [ "B" ] -> (Lista Circular de tamaño 2)
```

Imprimir $m=3$ desde $i=1$ (Posición $1 \bmod 2 = 1$):

1. Empezamos en pos 1 ("B"). Imprimimos "B".
2. Avanzamos: nos lleva a head. Lo saltamos y caemos en "A". Imprimimos "A".
3. Avanzamos: caemos en "B". Imprimimos "B".

Salida: B A B

Ejercicio 9: Suma de Elementos de una Lista

Archivo de código: [09-SumaLista.cpp]

(file:///c:/Users/AARON/Documents/GitHub/Guia_Ejercicios/Algoritmos_Y_Estructura_De_Datos/5-Listas_Y_Pilas/09-SumaLista.cpp) y método **suma()** en **ListaSimple.h**.

Enunciado

Realice una función 'suma' que retorne la suma de todos los elementos de una lista dada por parámetro.

Concepto y Algoritmo

La solución explota las propiedades del Tipo de Dato Abstracto (TDA) Lista. Para cumplir con el encapsulamiento del diseño base, se implementa el método **int suma()** dentro de la estructura base **intList** en [ListaSimple.h]

(file:///c:/Users/AARON/Documents/GitHub/Guia_Ejercicios/Algoritmos_Y_Estructura_De_Datos/9-Estructuras_Base/ListaSimple.h), mientras que el archivo de ejercicio provee la envoltura externa para el cliente.

Pasos del Algoritmo:

1. Inicializamos un acumulador **total = 0**.
2. Empezamos con un puntero temporal **current = head->next** (el primer nodo con datos válidos).
3. Mediante un ciclo **while**, recorremos la lista mientras **current != tail** (el nodo centinela final).
4. En cada paso, sumamos el valor del nodo actual al acumulador: **total += current->value**.
5. Avanzamos el puntero al siguiente nodo: **current = current->next**.
6. Al finalizar el ciclo, retornamos **total**.

Ejercicio 12: Eliminar Valores Repetidos de una Lista

Archivo de código: [12-EliminarRepetidos.cpp]

(file:///c:/Users/AARON/Documents/GitHub/Guia_Ejercicios/Algoritmos_Y_Estructura_De_Datos/5-Listas_Y_Pilas/12-EliminarRepetidos.cpp) y método `eliminarRepetidos()` en `ListaSimple.h`.

Enunciado

Cree una función que elimine de una lista simplemente enlazada de enteros, los valores repetidos.

Concepto y Algoritmo

Este problema requiere eliminar cualquier duplicado de un valor que aparezca en posiciones posteriores dentro de la lista, conservando únicamente la primera ocurrencia del elemento.

Pasos del Algoritmo:

1. Usamos un puntero `current` que arranca en el primer nodo de datos de la lista.
2. Para cada nodo apuntado por `current`, extraemos su valor (`valorBuscar = current->value`).
3. Inicializamos un segundo puntero `runner` que inicia justo después de `current` (`runner = current->next`).
4. Con `runner`, recorremos todo el resto de la lista.
5. Si encontramos que `runner->value == valorBuscar`:
 - Guardamos temporalmente la dirección del nodo duplicado: `temp = runner`.
 - Avanzamos el puntero `runner` al siguiente nodo de forma segura: `runner = runner->next`.
 - Eliminamos físicamente el nodo `temp` llamando a la primitiva de eliminación `Delete(temp)`. Esta primitiva reconecta el nodo anterior con el nuevo siguiente del nodo borrado y libera su memoria.
6. Si no es duplicado, simplemente avanzamos: `runner = runner->next`.
7. Cuando `runner` llega a `tail`, avanzamos `current` a su siguiente nodo y repetimos el proceso hasta llegar al final.

Visualización de Reconexión de Punteros:

```
Antes:    ... -> [ 20 ] -> [ 10 (runner) ] -> [ 30 ] -> ...
Duplicado de (10) encontrado.
Proceso:  1. runner se desplaza a [ 30 ].
           2. El nodo anterior [ 20 ] se conecta directamente a [ 30 ].
           3. Se libera la memoria de [ 10 ].
Despues:  ... -> [ 20 ] -----> [ 30 ] -> ...
```

Ejercicio 15: Inversión de Lista Enlazada (Iterativo y Recursivo)

Archivo de código: [15-InvertirLista.cpp]

(file:///c:/Users/AARON/Documents/GitHub/Guia_Ejercicios/Algoritmos_Y_Estructura_De_Datos/5-Listas_Y_Pilas/15-InvertirLista.cpp) y métodos `reverseIterative()` / `reverseRecursive()` en `ListaSimple.h`.

Enunciado

Elabore dos algoritmos (uno recursivo y otro iterativo) en el cual dada una lista lineal en forma enlazada la invierta, sin crear una nueva lista, ni mover los elementos físicamente de la lista.

Concepto e Implementación

La restricción crítica de este ejercicio es **"sin crear una nueva lista, ni mover los elementos físicamente"**. Esto implica que no podemos intercambiar los valores internos de los nodos, ni crear nuevos nodos con `new`. Debemos **redirigir los punteros `next`** existentes en la lista para que los nodos de datos se enlacen en sentido contrario, reubicando los centinelas inicial (`head`) y final (`tail`).

A) Algoritmo Iterativo (`reverseIterative`)

Utiliza tres punteros auxiliares para avanzar por la lista sin perder la referencia a los nodos mientras se realiza la inversión de enlaces:

1. `prev`: Apunta al nodo que debe quedar como siguiente del nodo actual. Inicialmente es `tail` (porque el primer nodo de la lista original, al invertirse, pasará a apuntar a `tail`).
2. `curr`: Nodo actual que se está invirtiendo. Comienza en `head->next` (el primer dato).
3. `next_node`: Guarda el enlace al siguiente nodo original antes de romper la flecha.

Ciclo de Inversión:

```
while (curr != tail) {
    Node* next_node = curr->next; // Guarda el siguiente original
    curr->next = prev;             // Invierte el enlace
    prev = curr;                  // Mueve prev un paso adelante
    curr = next_node;             // Avanza curr
}
head->next = prev;                // head ahora apunta al último nodo original
(nuevo primero)
```

B) Algoritmo Recursivo (`reverseRecursive`)

Utiliza la pila de llamadas de la recursión para recorrer la lista hasta el final y, al retornar, ir invirtiendo las flechas de atrás hacia adelante:

1. **Caso Base:** Si la lista está vacía (`curr == tail`) o tiene un solo elemento (`curr->next == tail`), retornamos el nodo directamente.
2. **Paso Recursivo:**
 - Llamamos recursivamente con el siguiente nodo: `Node* new_head = reverse_rec(curr->next)`. Esto recorre toda la lista hasta llegar al último nodo de datos y nos retorna la nueva cabeza de la lista invertida.
 - Al regresar de la llamada, hacemos que el nodo siguiente original apunte de regreso a nosotros: `curr->next->next = curr`.
 - Hacemos que nuestro nodo actual apunte temporalmente a la cola para evitar ciclos infinitos: `curr->next = tail`.
 - Retornamos `new_head` hacia la llamada anterior.

Visualización del Retorno de la Recursión:

```
Llamada con curr = D2:  
D2 —> D3 —> tail  
Después de invertir recursivamente D3:  
D2 —> D3 (new_head)  
Reconexión:  
D2 <— D3 (curr->next->next = curr)  
D2 —> tail (curr->next = tail)
```

Ejercicio 17: Inversión de una Pila mediante Recursión

Archivo de código: [17-InvertirPila.cpp]

(file:///c:/Users/AARON/Documents/GitHub/Guia_Ejercicios/Algoritmos_Y_Estructura_De_Datos/5-Listas_Y_Pilas/17-InvertirPila.cpp) y método `invertir()` en `PilaSimple.h`.

Enunciado

Utilizando únicamente las primitivas de la clase Pila, se quiere que Ud. desarrolle un procedimiento que dada una pila P, la invierta. No debe utilizar estructuras auxiliares.

Concepto y Algoritmo

La inversión de una pila usando exclusivamente sus primitivas (`Push`, `Pop`, `IsEmpty`) y sin estructuras auxiliares (como colas u otras pilas) se resuelve mediante un **doble proceso recursivo**. La pila de llamadas del sistema (call stack) actúa como almacenamiento temporal para los datos extraídos de la pila original.

Algoritmo de Inversión (`invertir`)

1. Si la pila está vacía, finalizamos la función (caso base).
2. De lo contrario, extraemos el elemento del tope y lo guardamos en una variable local del call stack: `int temp = Pop()`.
3. Llamamos recursivamente a `invertir()`. Esto vaciará e invertirá toda la pila restante.
4. Una vez invertida la sub-pila, insertamos el elemento guardado `temp` en el fondo de la pila llamando a la función auxiliar `insertAtBottom(temp)`.

Algoritmo Auxiliar de Inserción en el Fondo (`insertAtBottom`)

Esta función recursiva coloca un elemento en la base misma de la pila:

1. Si la pila está vacía, simplemente apilamos el elemento: `Push(item)` (caso base).
2. Si no está vacía:
 - Removemos el elemento del tope actual: `int temp = Pop()`.
 - Llamamos recursivamente a `insertAtBottom(item)` para colocar el elemento original en la base de la pila restante.
 - Volvemos a colocar el elemento removido en el tope: `Push(temp)`.

Simulación del Call Stack para insertAtBottom(9) en Pila [4, 3]:

```
Llamada 1: Pop de 4. Pila queda: [3]. Llama a insertAtBottom(9)
  Llamada 2: Pop de 3. Pila queda: []. Llama a insertAtBottom(9)
    Llamada 3: Pila vacía. Push(9). Pila queda: [9]. Retorna.
  Llamada 2 (Retorno): Push(3). Pila queda: [3, 9] (tope a base). Retorna.
Llamada 1 (Retorno): Push(4). Pila queda: [4, 3, 9] (tope a base). Finalizado.
```